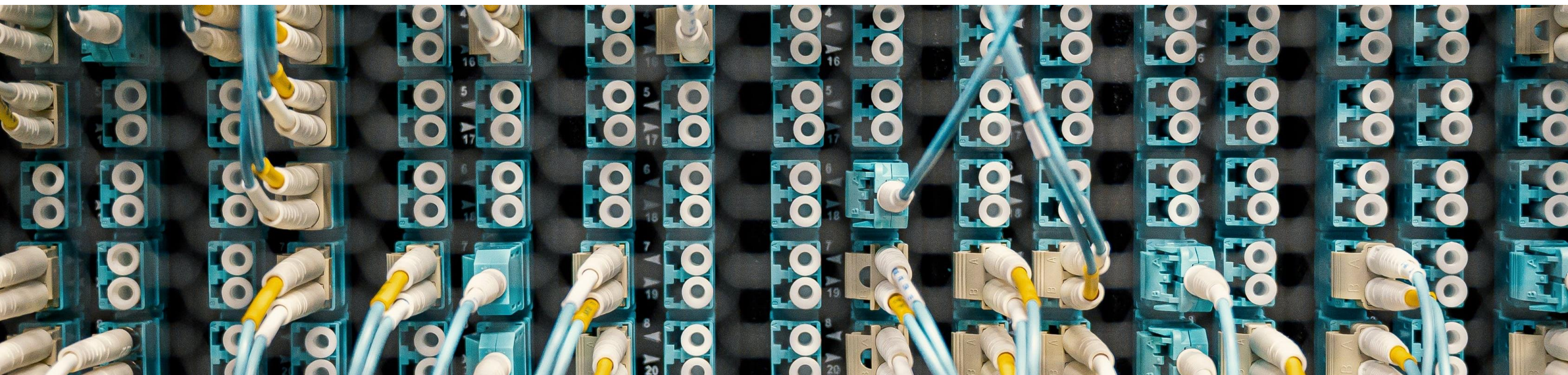




Artificial Intelligence in Interactive Systems

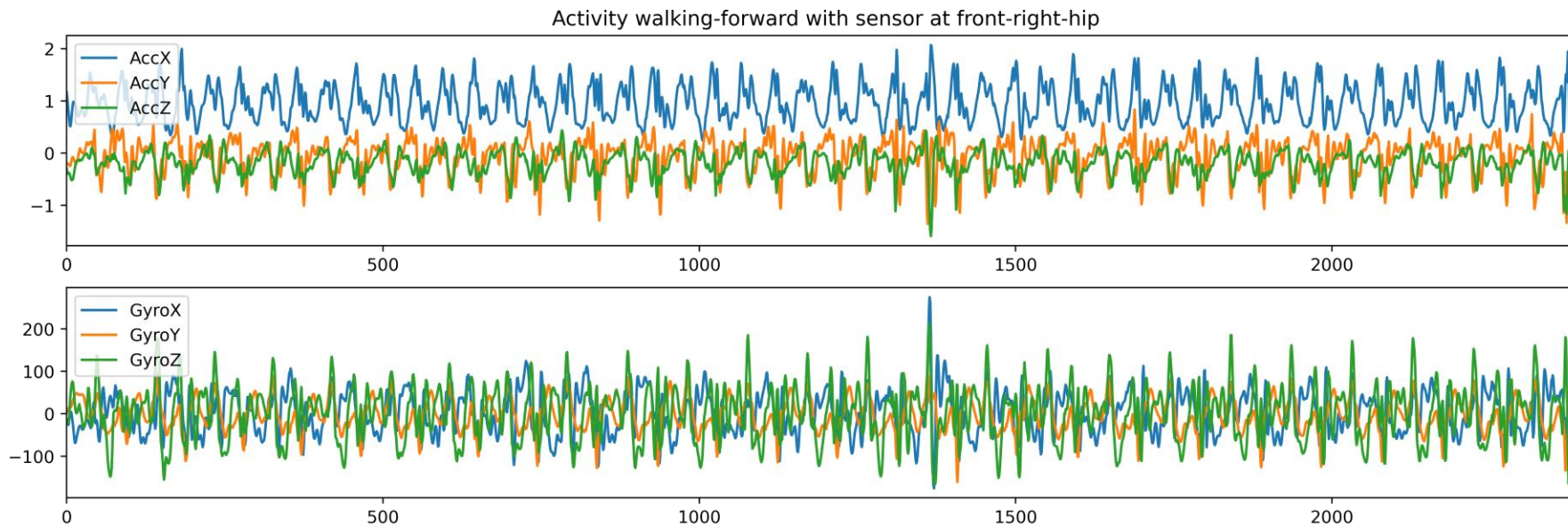
Recurrent Neural Network and Long Short-Term Memory

Model Layers

- Dense Layer
 - No specific relationship between inputs
- CNN Layer
 - Building an understanding by looking at neighboring inputs in an n-dimensional input

Sequence Data

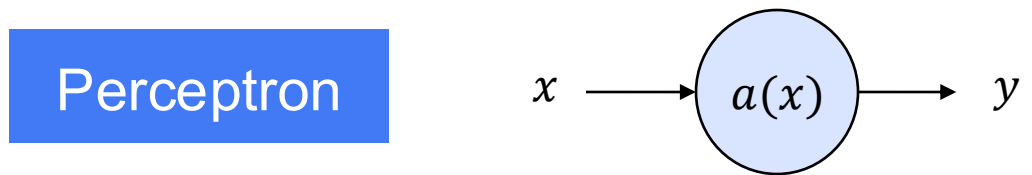
- Time Series Data
- Natural Language



Data Source: <https://dl.acm.org/doi/10.1145/2370216.2370438>

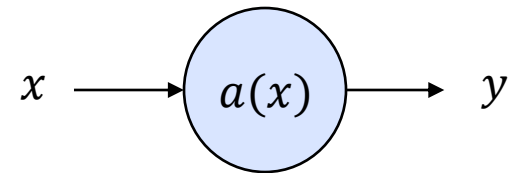
Recurrent Neural Network (RNN)

Recap Perceptron

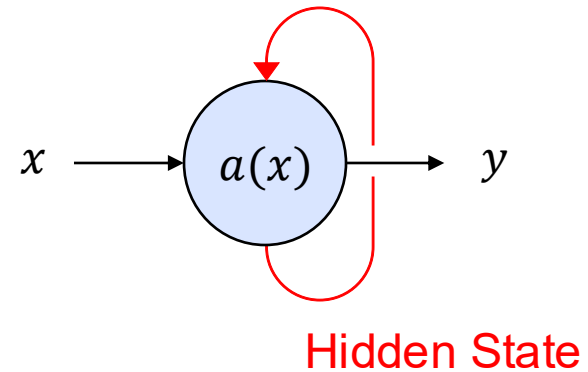


Recurrent Neural Network (RNN)

Perceptron

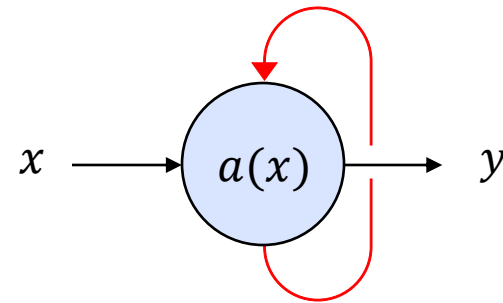
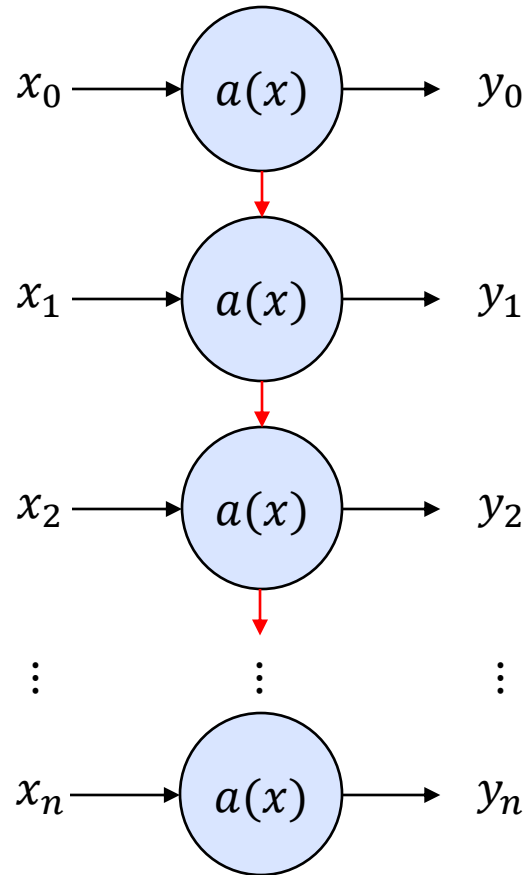


RNN Unit

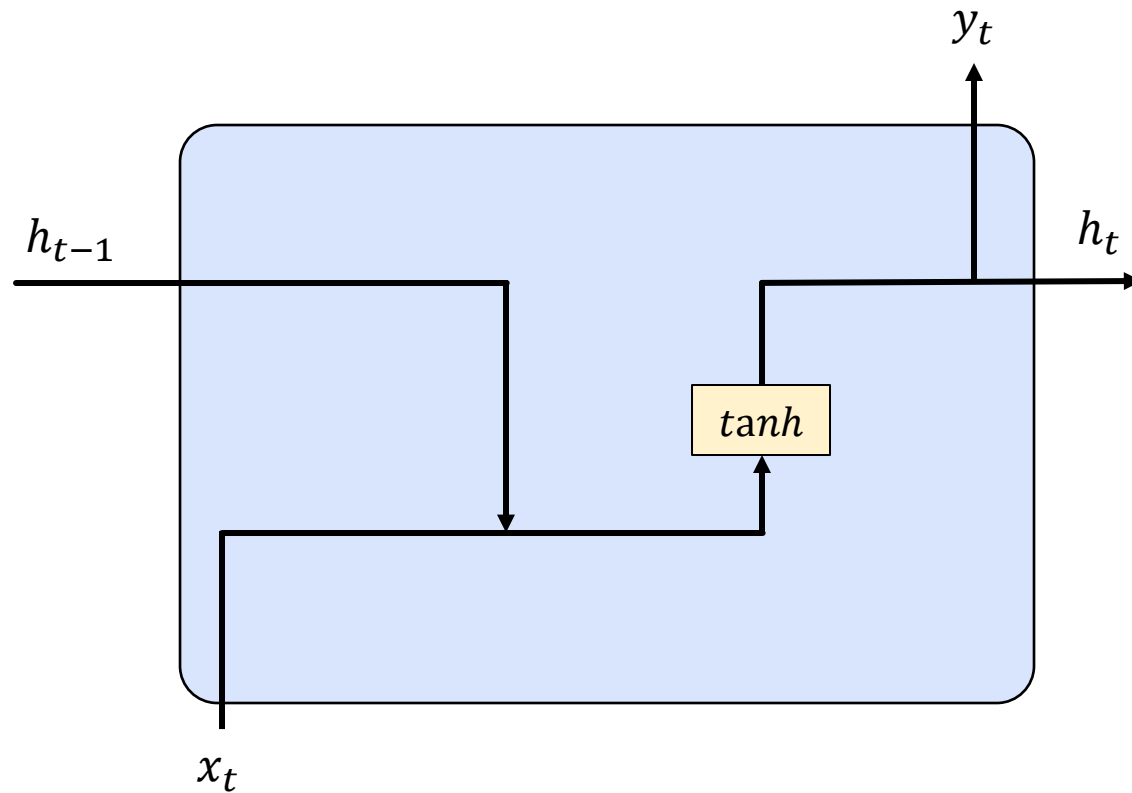


Recurrent Neural Network

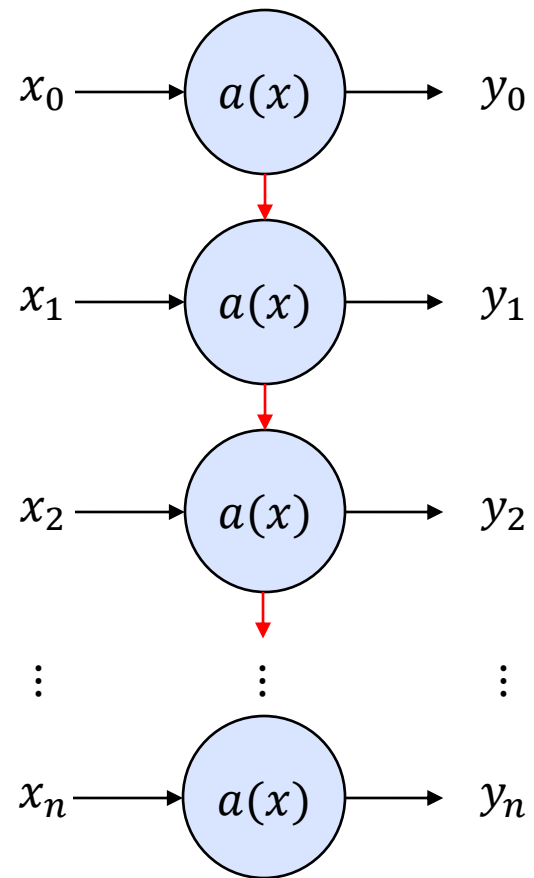
Unrolled Recurrent Neural Network



Recurrent Neural Network



Recurrent Neural Network



- Each state gets the “recent” information
- Long-term dependencies gets lost

RNN in TensorFlow

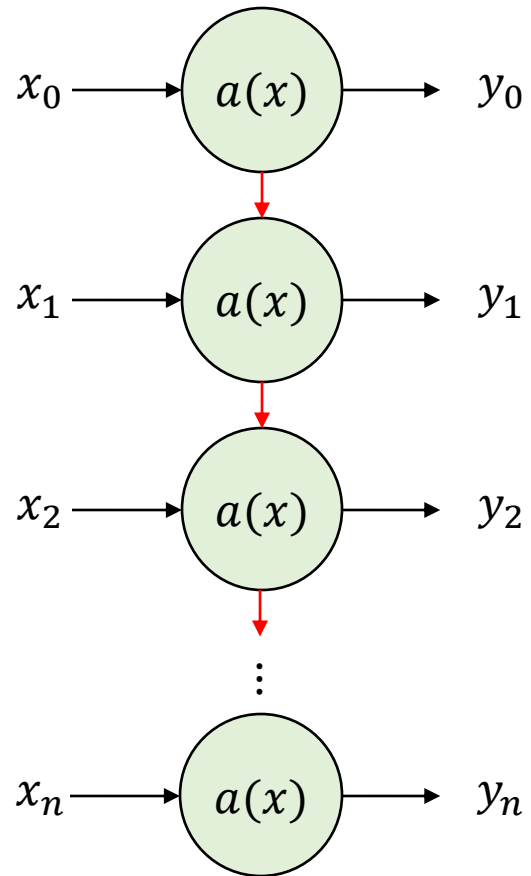
Two RNN layer using a fixed window length

```
inputs = Input(shape=(WINDOW_LENGTH, 3), name="Input")
x = SimpleRNN(32, name="RNN_1", return_sequences=True)(inputs)
x = SimpleRNN(32, name="RNN_2")(x)
prediction = Dense(len(classes), activation="softmax", name="Output")(x)
model_functional = tf.keras.Model(inputs = inputs, outputs = prediction)
model_functional.summary()
```

Layer (type)	Output Shape	Param #
Input (InputLayer)	[(None, 200, 3)]	0
RNN_1 (SimpleRNN)	(None, 200, 32)	1152
RNN_2 (SimpleRNN)	(None, 32)	2080
Output (Dense)	(None, 12)	396
Total params: 3,628		
Trainable params: 3,628		
Non-trainable params: 0		

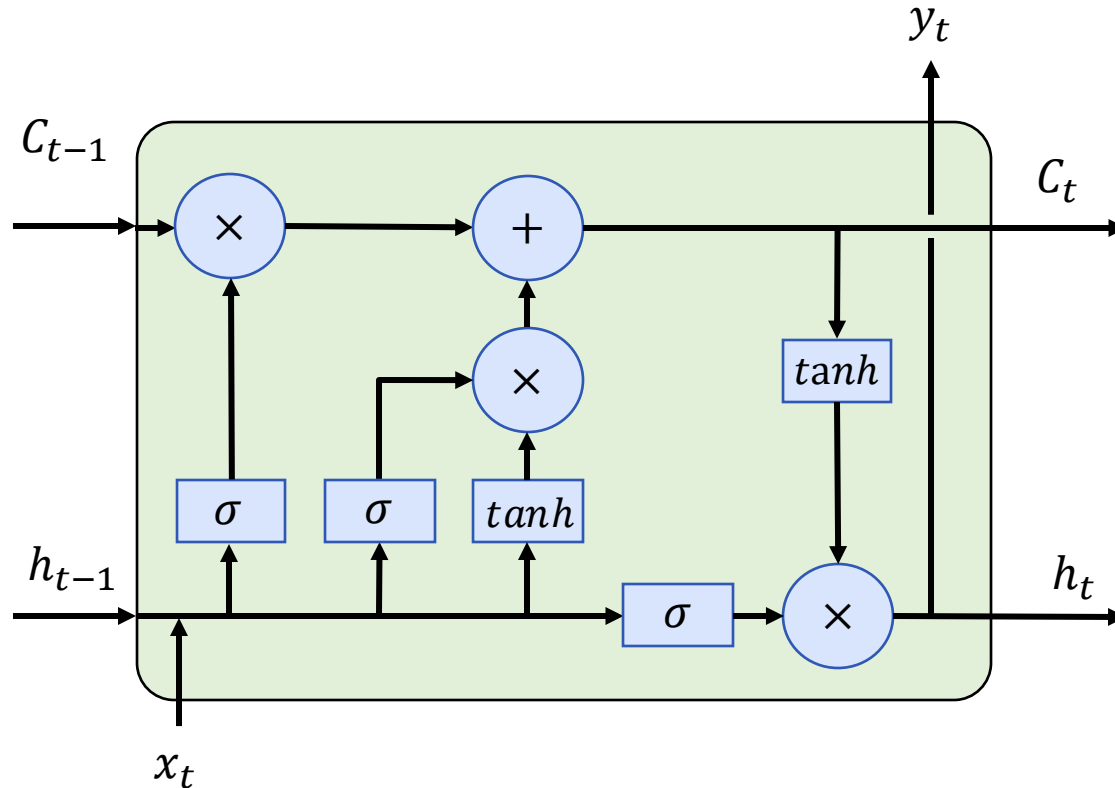
Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM)



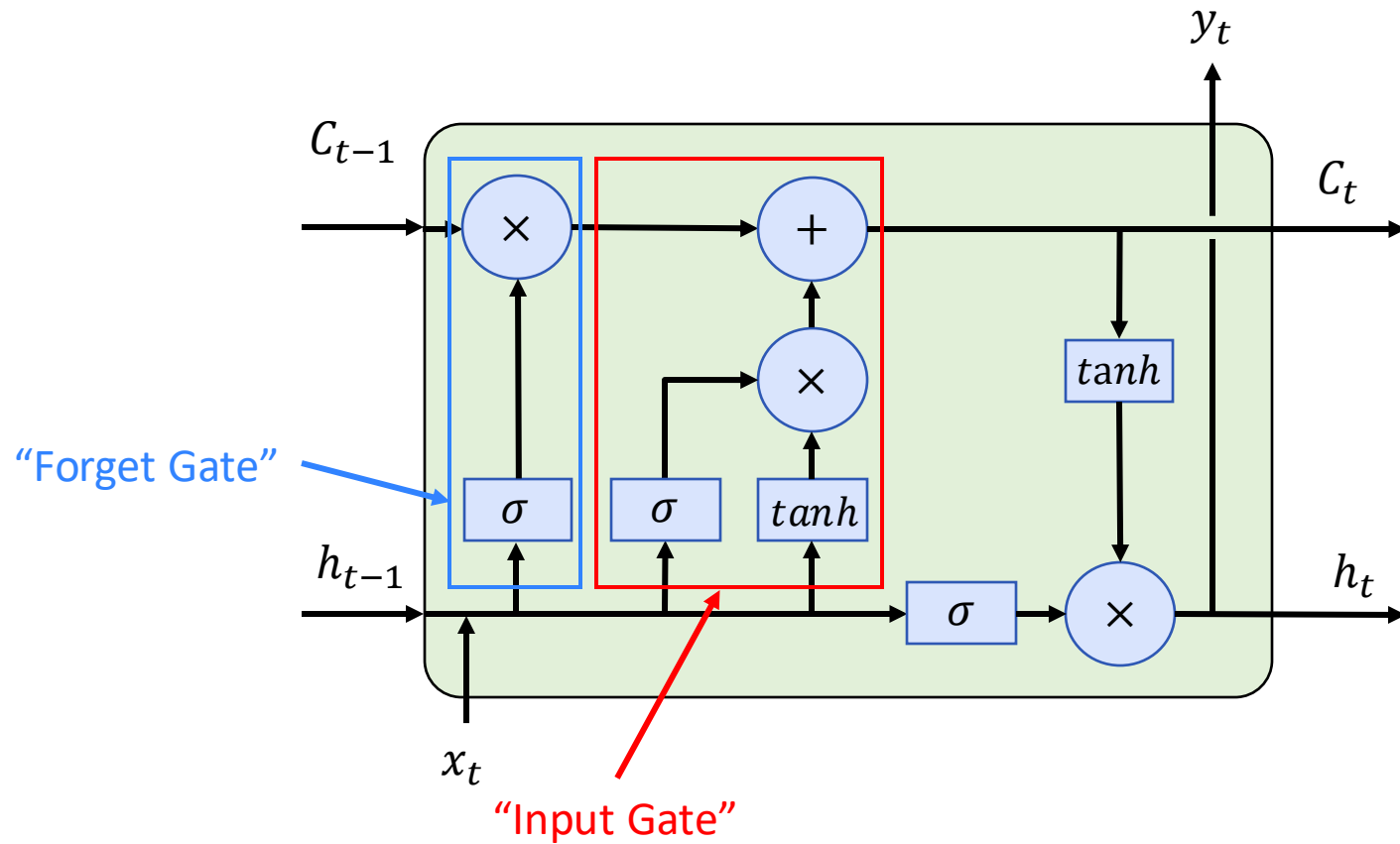
Sepp Hochreiter, and Schmidhuber Jürgen. "Long short-term memory." *Neural computation* (1997).

Long Short-Term Memory (LSTM)



Credit: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long Short-Term Memory (LSTM)



LSTM in TensorFlow

Two RNN layer using a fixed window length

```
inputs = Input(shape=(WINDOW_LENGTH, 3), name="Input")
x = LSTM(32, name="LSTM_1", return_sequences=True)(inputs)
x = LSTM(32, name="LSTM_2")(x)
prediction = Dense(len(classes), activation="softmax", name="Output")(x)
model_functional = tf.keras.Model(inputs = inputs, outputs = prediction)
model_functional.summary()
```

Layer (type)	Output Shape	Param #
Input (InputLayer)	[(None, 200, 3)]	0
LSTM_1 (LSTM)	(None, 200, 32)	4608
LSTM_2 (LSTM)	(None, 32)	8320
Output (Dense)	(None, 12)	396

=====
Total params: 13,324
Trainable params: 13,324
Non-trainable params: 0

Conclusion

Recurrent Neural Network and Long Short-Term Memory

- Processing sequence data using neuronal networks
- Recurrent Neural Network (RNN)
- Long Short-Term Memory (LSTM)
- Variations
 - Gated Recurrent Unit (GRU) [1]

[1] Cho, Kyunghyun, et al. "Learning phrase representations using RNN encoder-decoder for statistical machine translation." *arXiv preprint arXiv:1406.1078* (2014).

License

This file is licensed under the Creative Commons
Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Sven Mayer

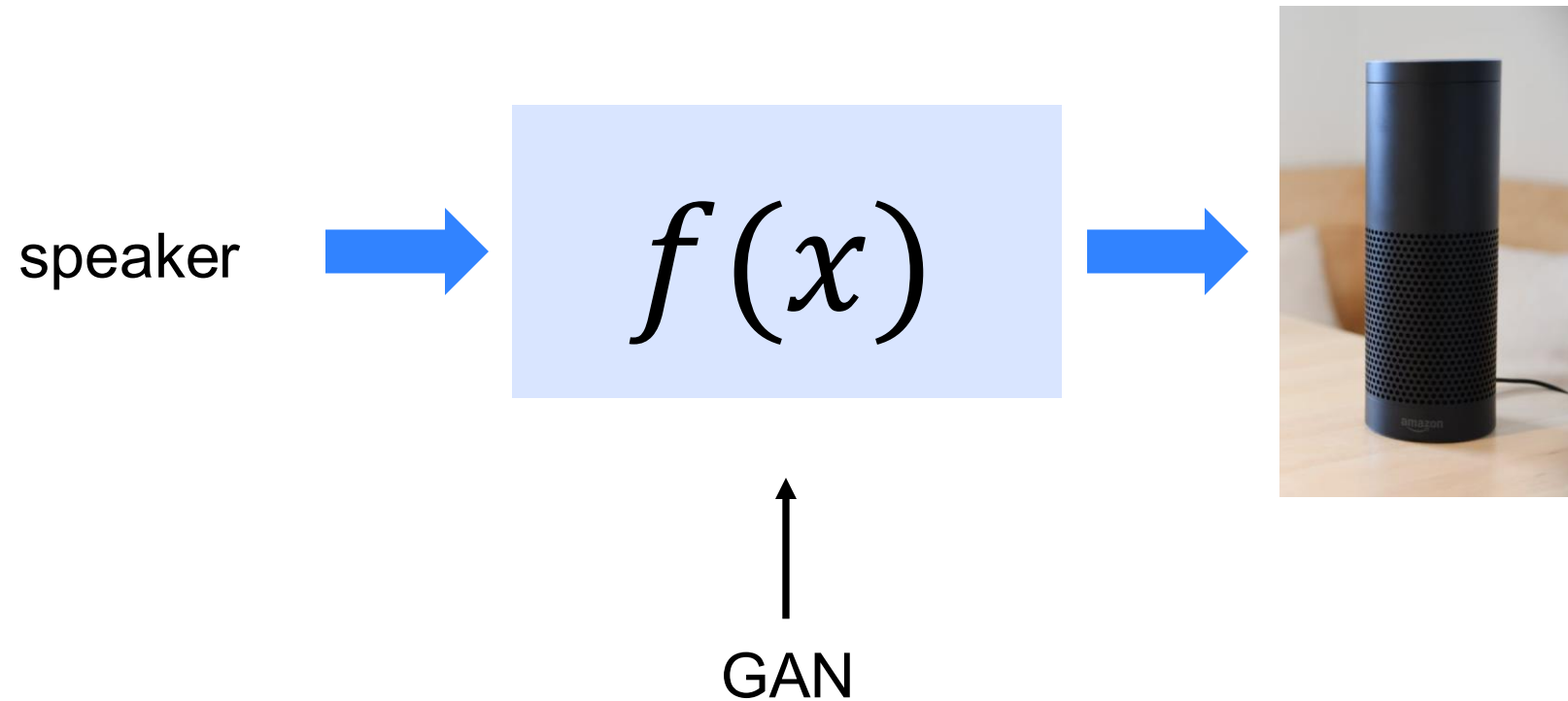




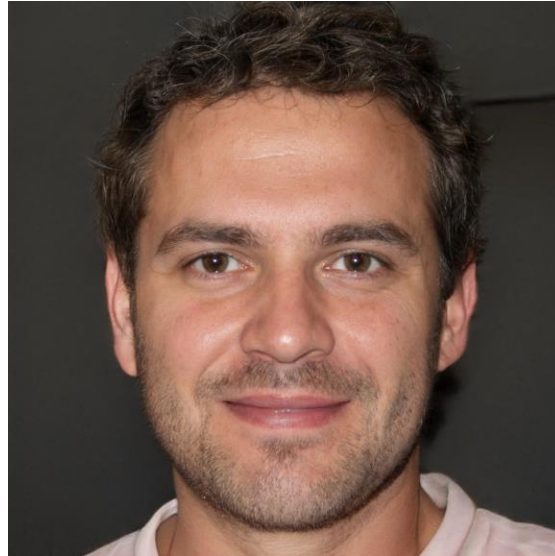
Artificial Intelligence in Interactive Systems

Generative Adversarial Network

Generative Adversarial Network (GAN)



Face Generation



Source: <https://thispersondoesnotexist.com/>

Karras, Tero, et al. "Analyzing and improving the image quality of stylegan." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020. DOI: <https://arxiv.org/abs/1912.04958>

Style Transfer

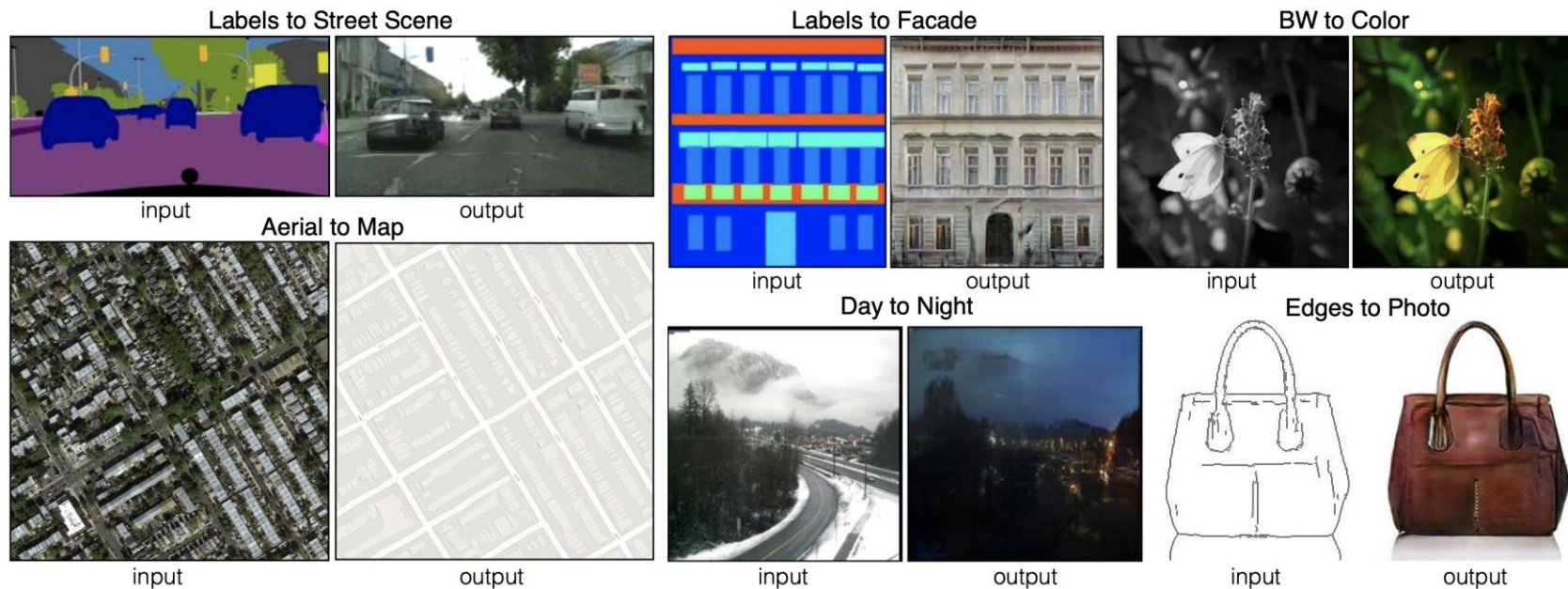


Figure 1: Many problems in image processing, graphics, and vision involve translating an input image into a corresponding output image. These problems are often treated with application-specific algorithms, even though the setting is always the same: map pixels to pixels. Conditional adversarial nets are a general-purpose solution that appears to work well on a wide variety of these problems. Here we show results of the method on several. In each case we use the same architecture and objective, and simply train on different data.

Isola, Phillip, et al. "Image-to-image translation with conditional adversarial networks." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017. DOI: <https://arxiv.org/abs/1611.07004>

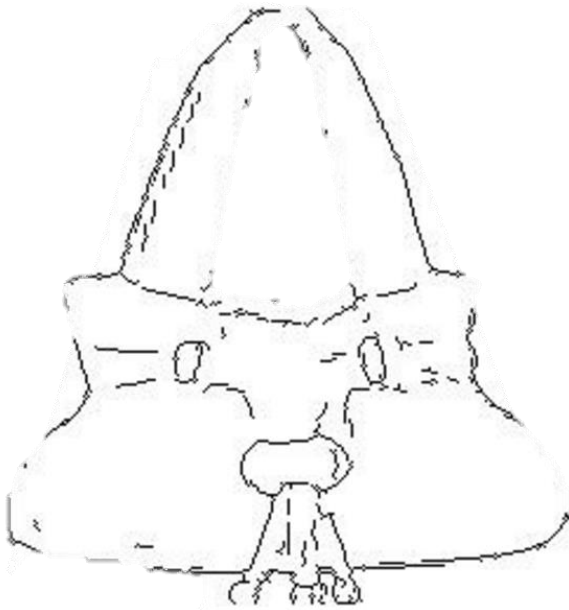
input



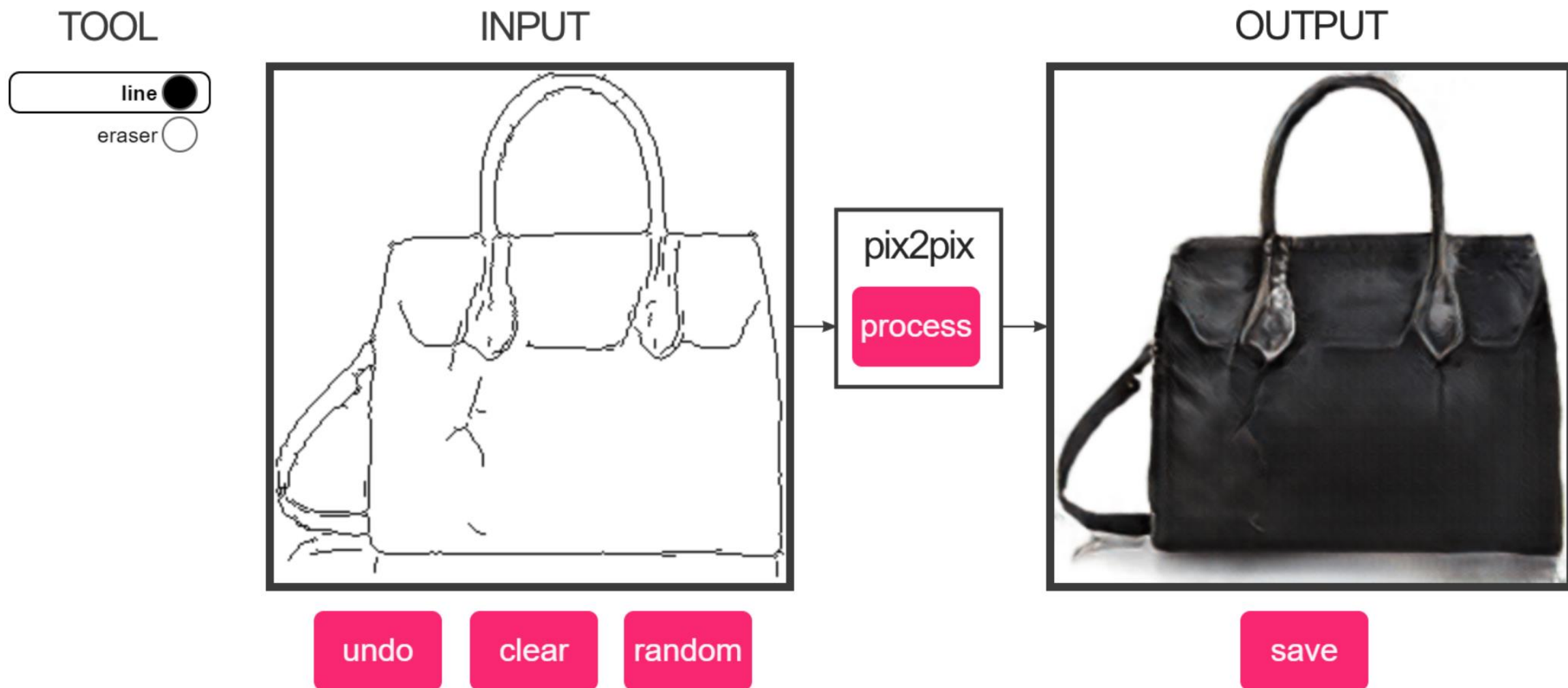
output



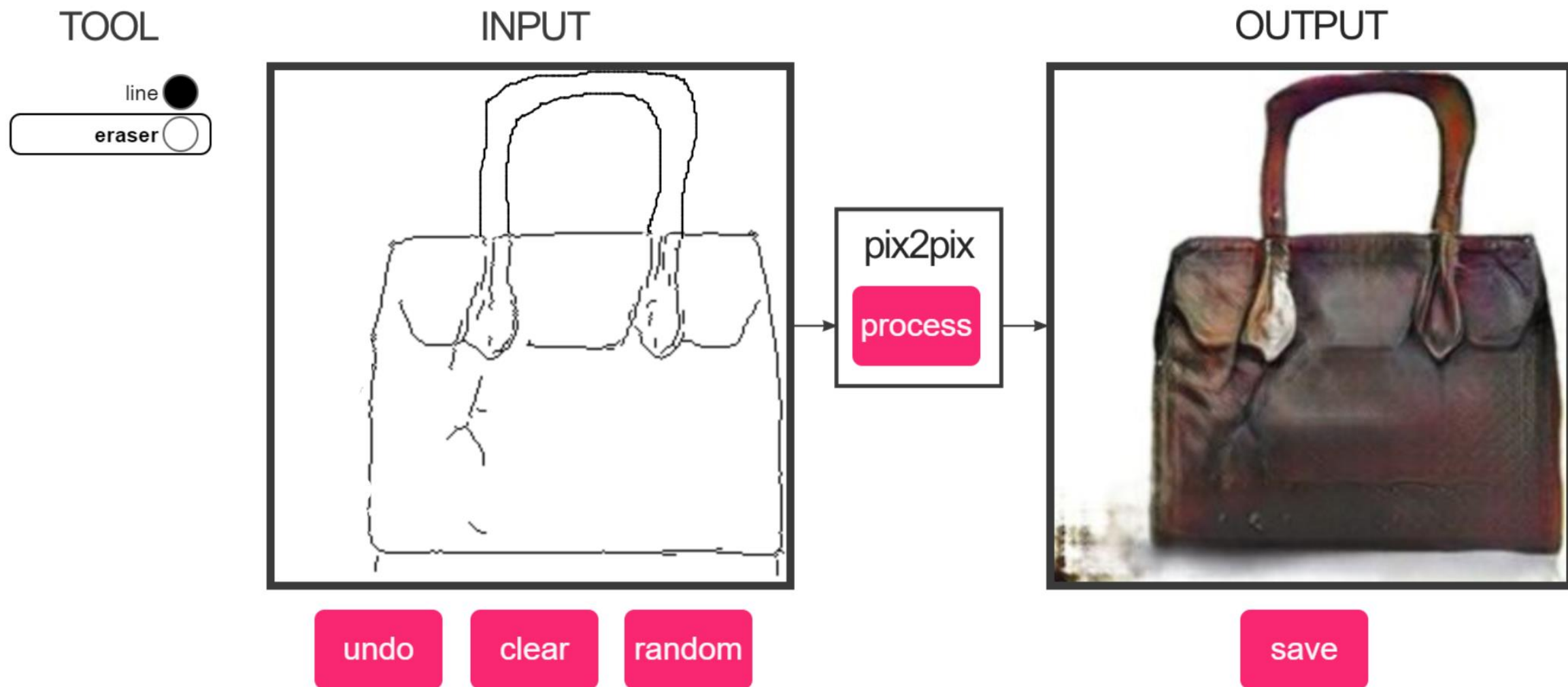
input

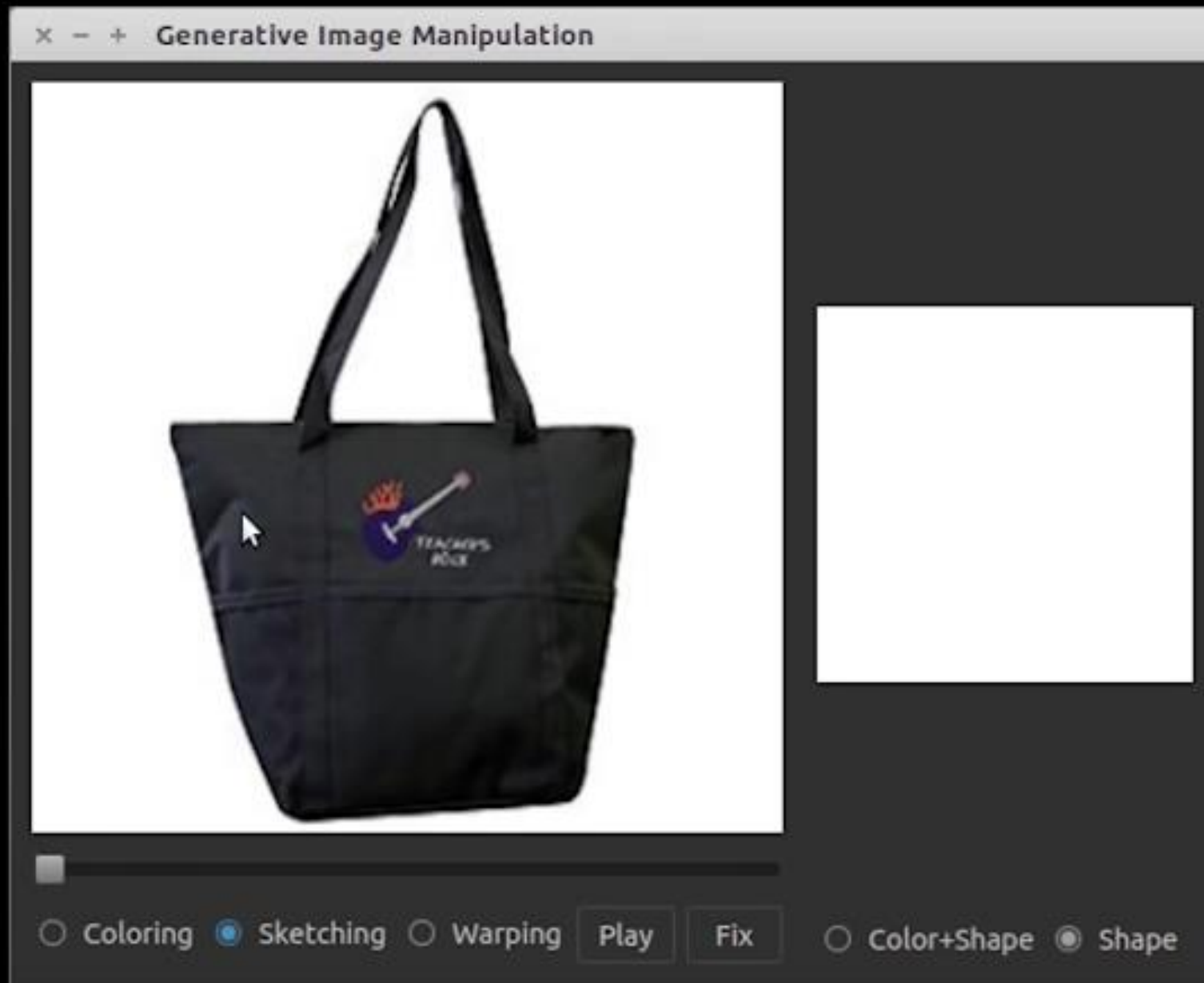


edges2handbags



edges2handbags

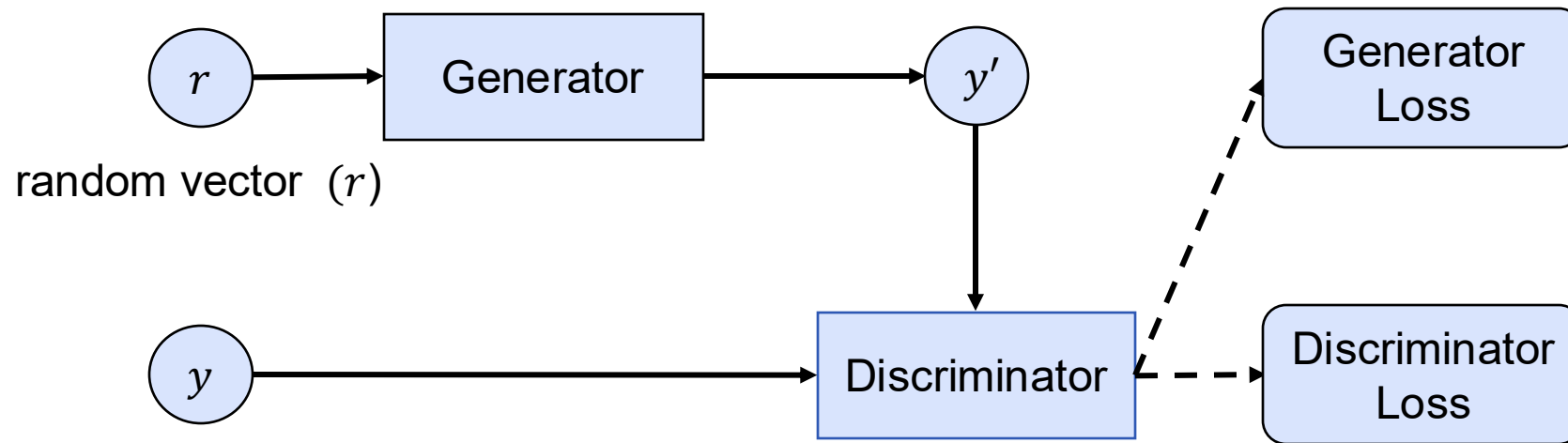




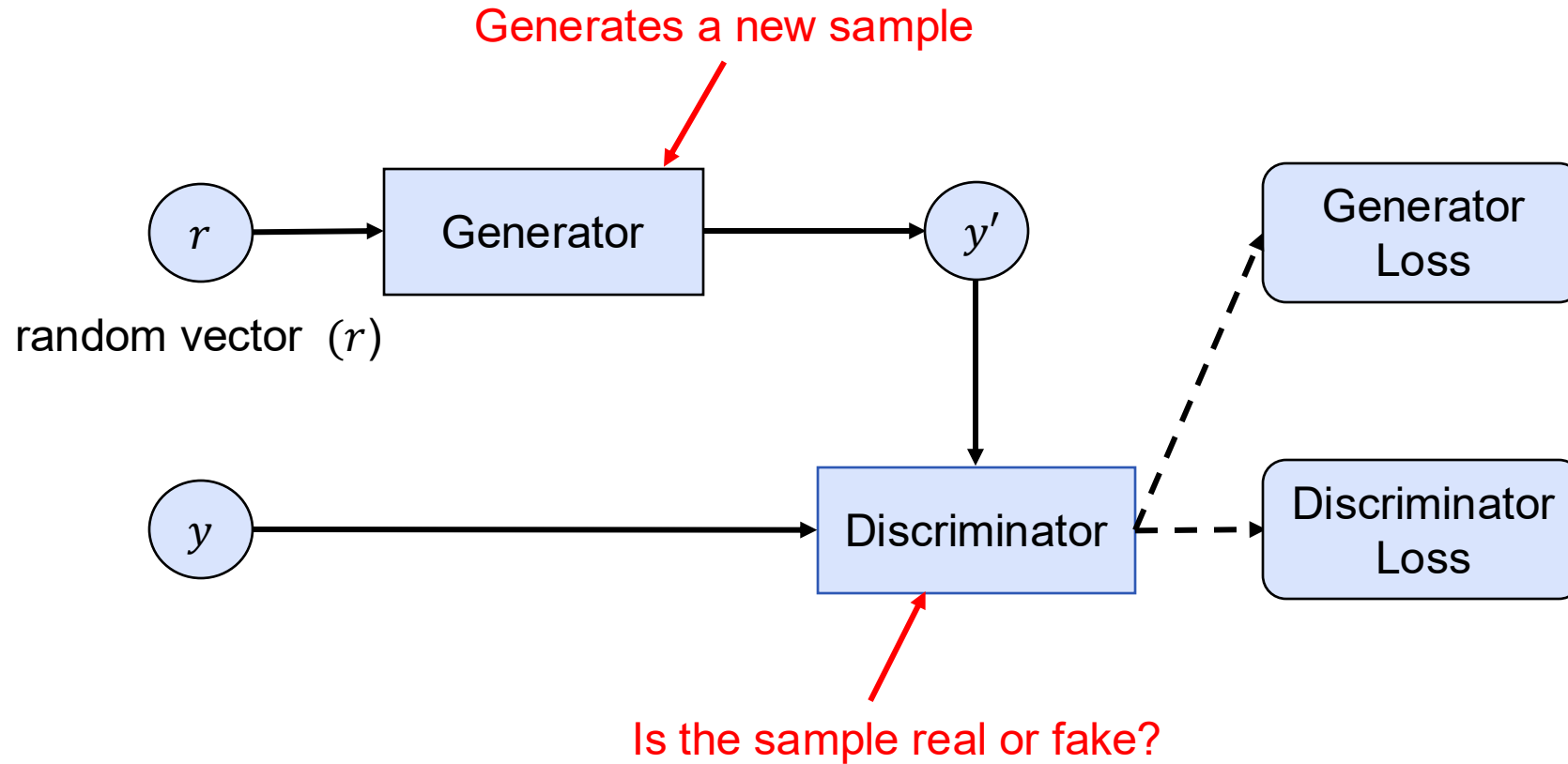
Generative Adversarial Network

- Create Data
- Style Transfer
- Increase Image Resolution (super-resolution)
- ...

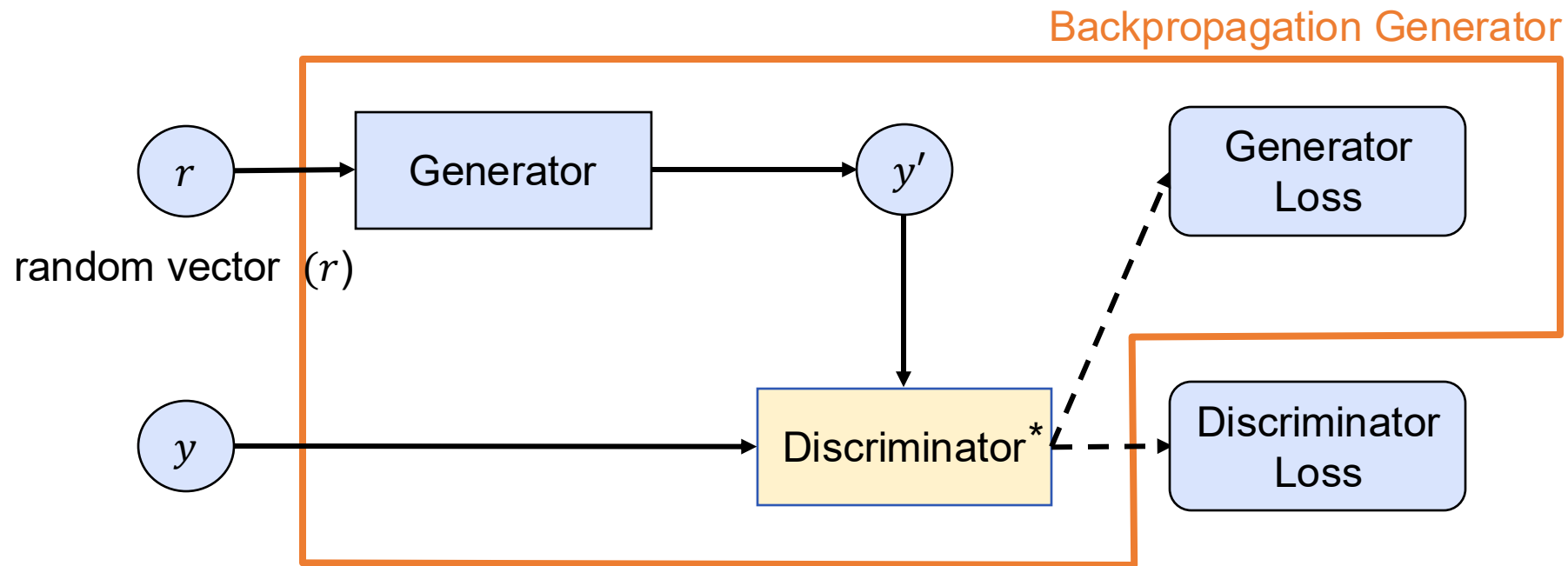
Generative Adversarial Network



Generative Adversarial Network

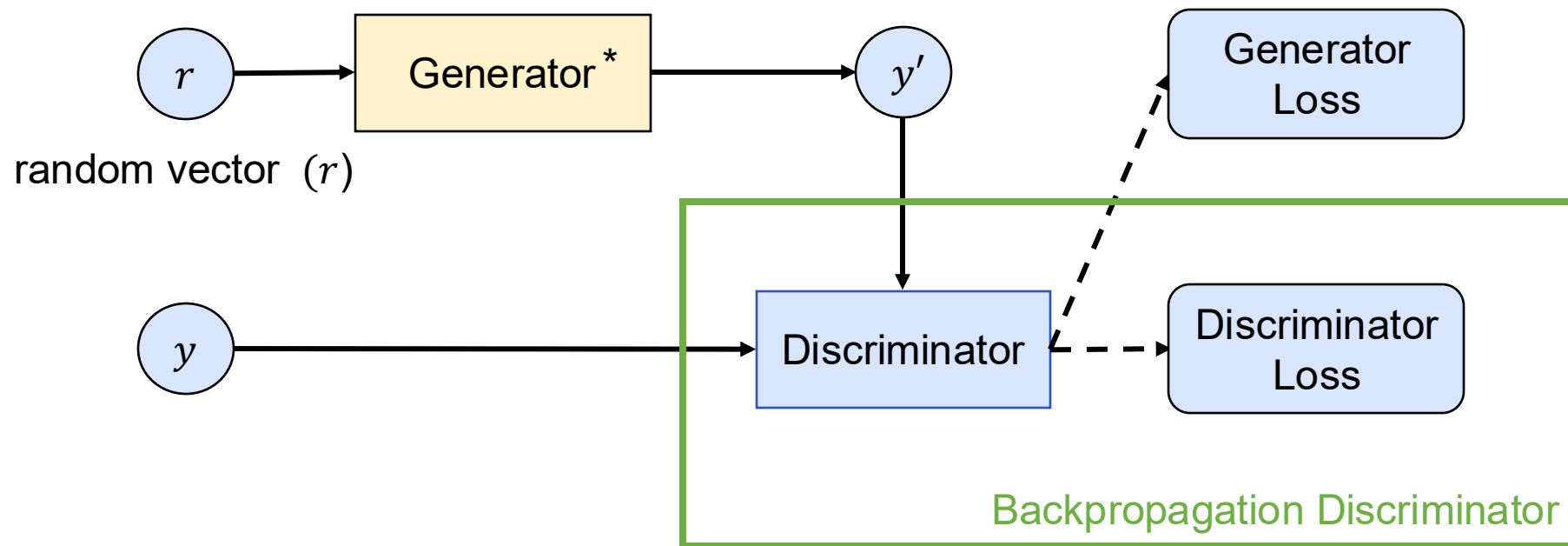


Generator Training



* Frozen model

Discriminator Training



* Frozen model

Training

During the training process the generator and the discriminator improve each other by both getting better each trying to outperform the other model.

What is a good Generator?

In the end the generator is should be so good to fool the discriminator. Thus, the discriminator needs to guess fake or real (accuracy is $\sim .5$).

Keynote Slides

 [mimuc](#) / [Conductive-Fiducial-Marker-Simulation-Toolkit](#) Public

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#)

 [main](#)

 1 branch

 0 tags

[Go to file](#) [Code](#)

 **sven-mayer** Merge branch 'main' of [https://github.com/mimuc/Conductive-...](#) [fe75506](#) on Jan 30  15 commits

 markers_new	added markers	last year
 markers_official	added markers	last year
 model_simulator	added pre-trained simulator model	last year
 .gitignore	Update .gitignore	last year
 CITATION.cff	Create CITATION.cff	last year
 LICENSE	Initial commit	last year
 README.md	Update README.md	last year
 Step_01_Download_Data.ipynb	added files	5 months ago
 Step_02_Simulator_training.ipynb	Create Step_02_Simulator_training.ipynb	last year
 Step_03_SimulatorEvaluation.ipynb...	Create Step_03_SimulatorEvaluation.ipynb	last year
 Step_04_RecognizerModel_train...	Create Step_04_RecognizerModel_training.ipynb	last year
 files.txt	added files	5 months ago
 process.py	added helper files	last year
 recognizer.py	added helper files	last year
 requirements.txt	added helper files	last year
 simulate.py	added helper files	last year

 **README.md**

Conductive-Fiducial-Marker-Simulation-Toolkit

About

No description, website, or topics provided.

 Readme

 MIT license

 Cite this repository

 0 stars

 9 watching

 0 forks

[Report repository](#)

Releases

No releases published

Packages

No packages published

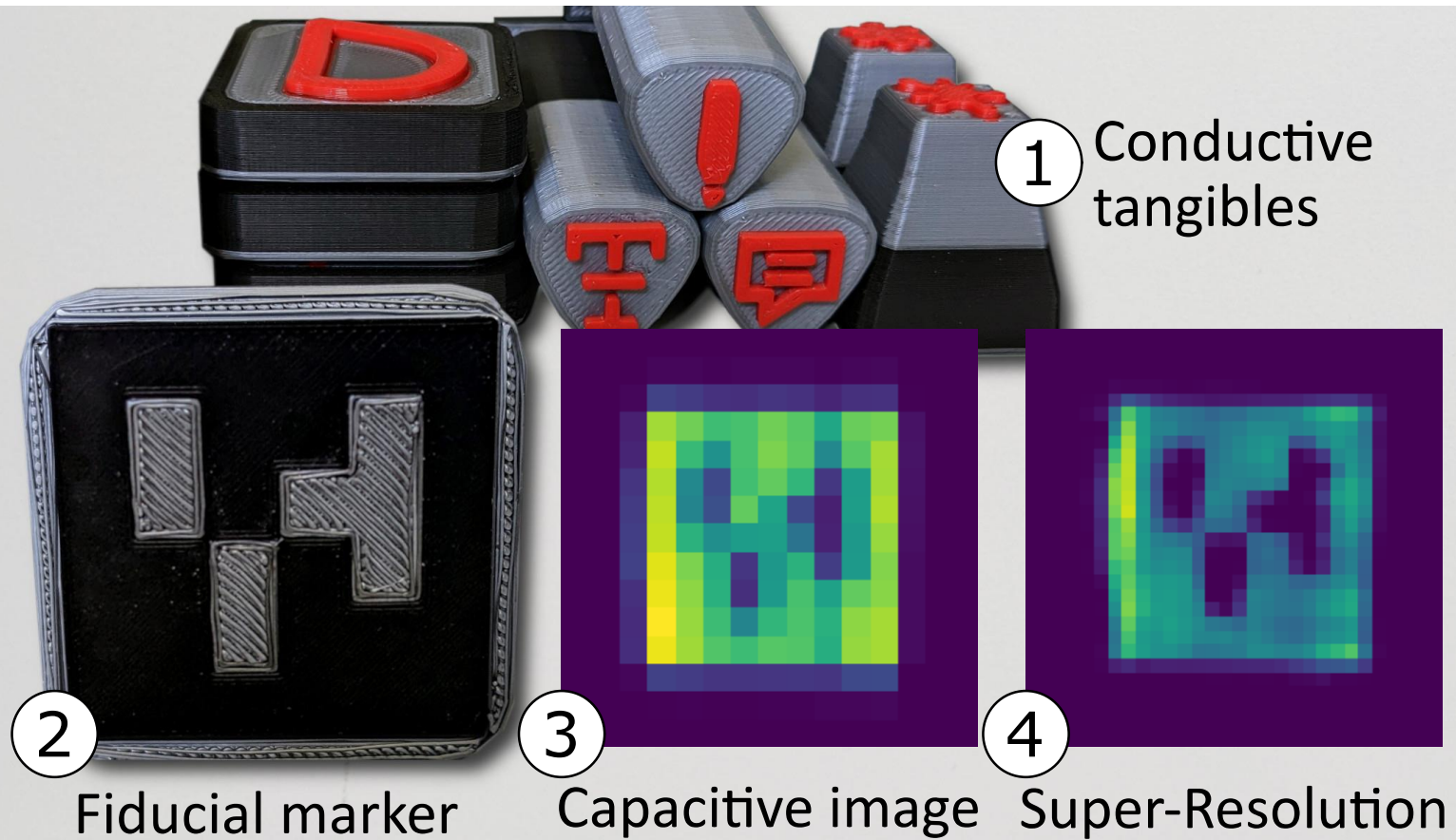
Languages

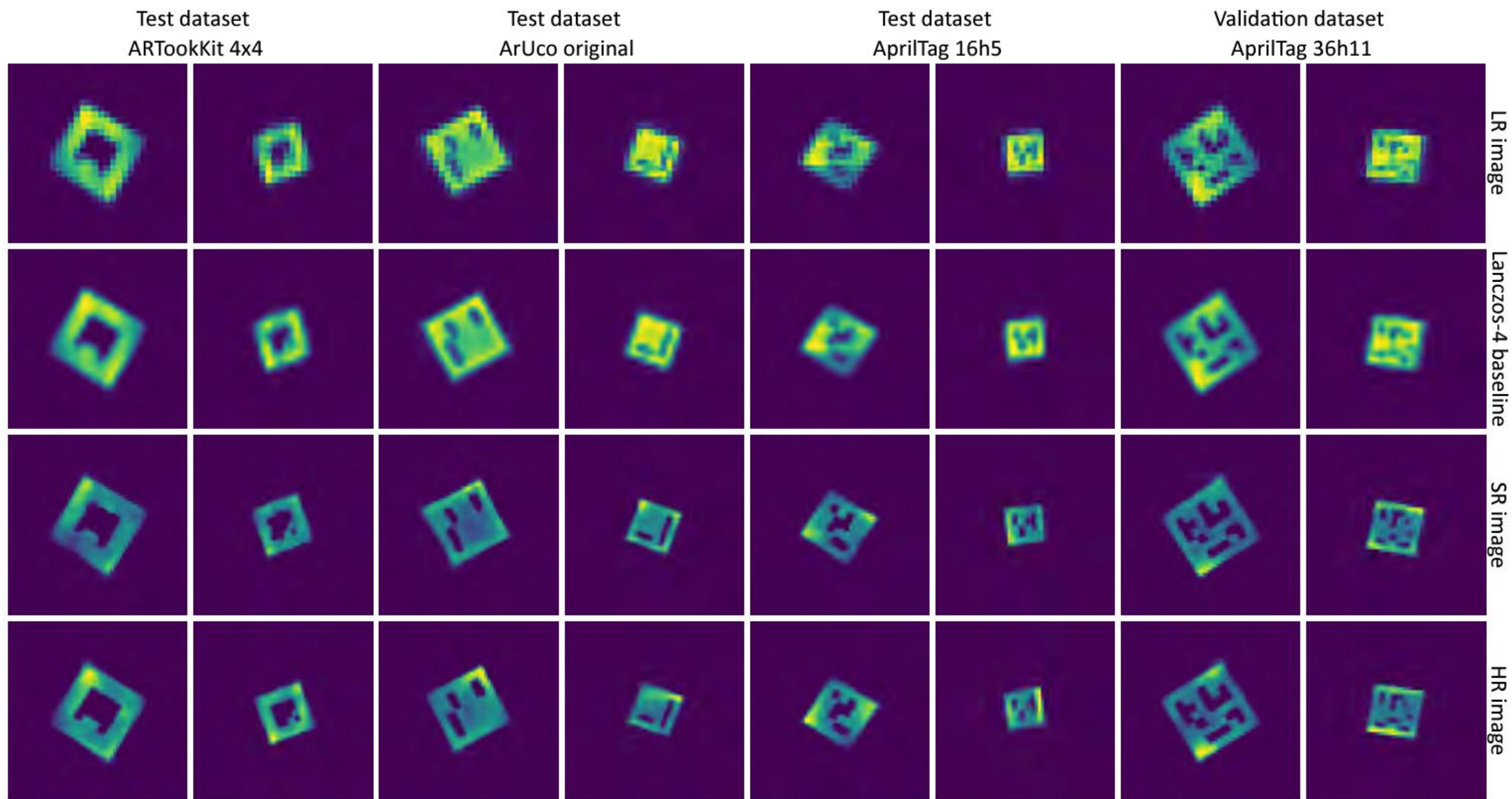
Jupyter Notebook 92.9%

PureBasic 5.9%

Python 1.2%

Super Resolution + GAN





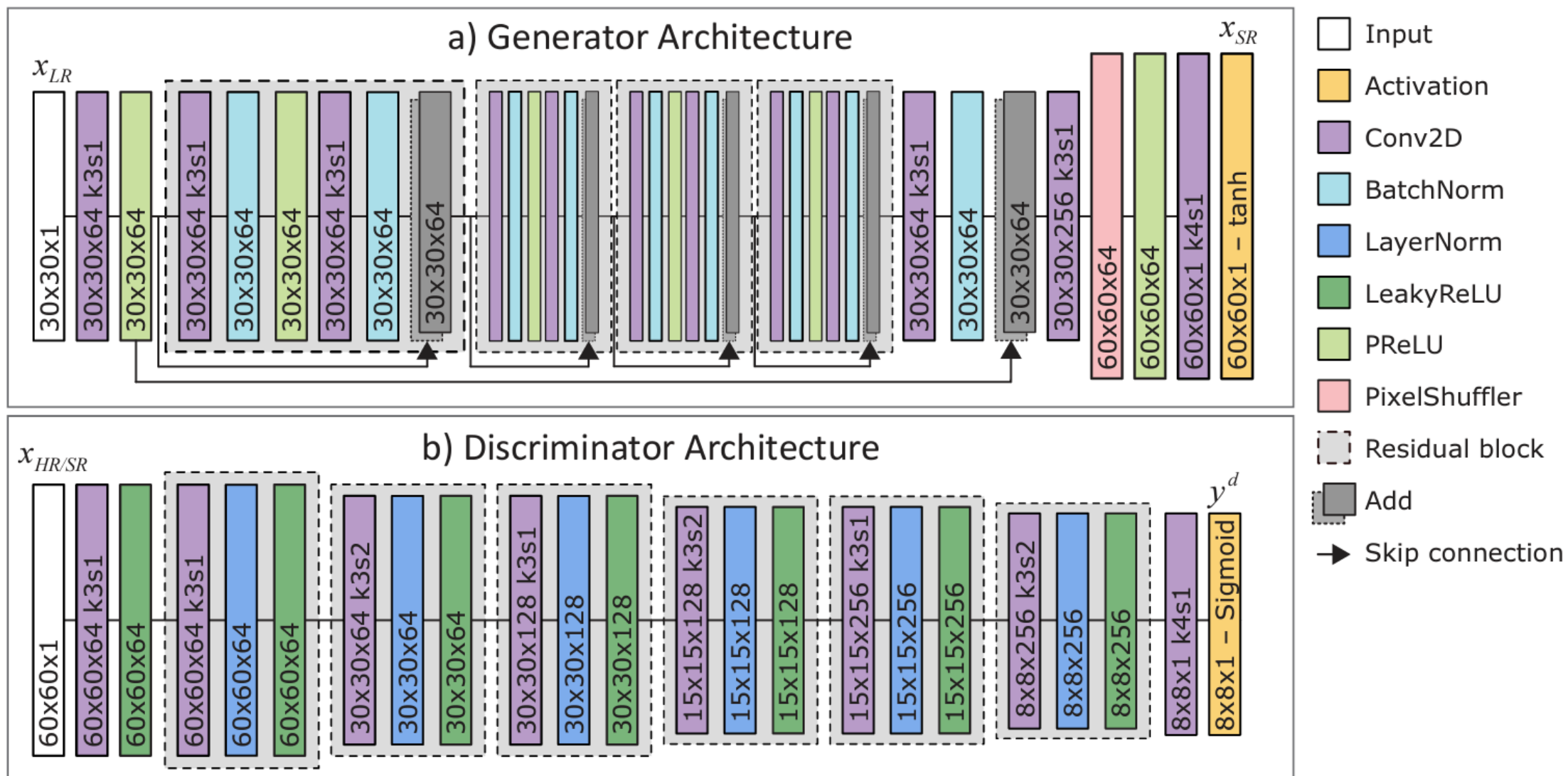


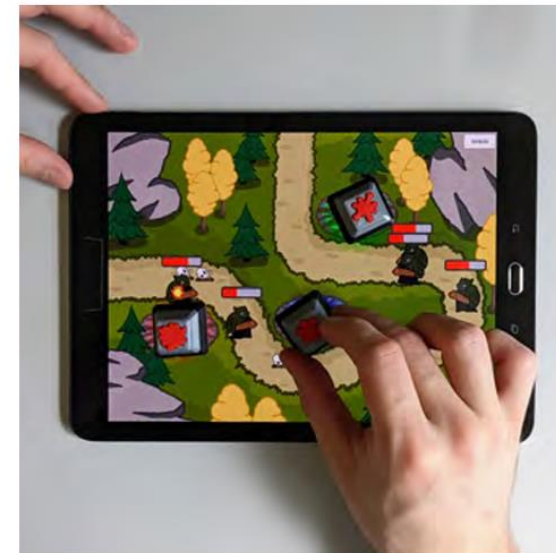
Figure 5: a) The architecture of the Generator with 1,002,433 parameters. b) The architecture of the Discriminator with 1,187,073 parameters.



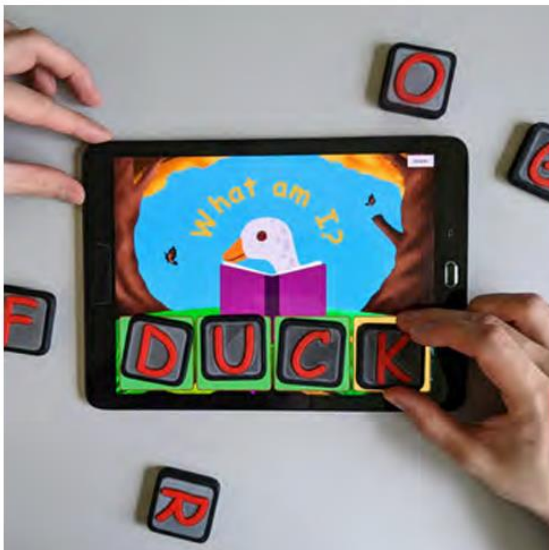
(a) Fabricated multi-material tangibles.



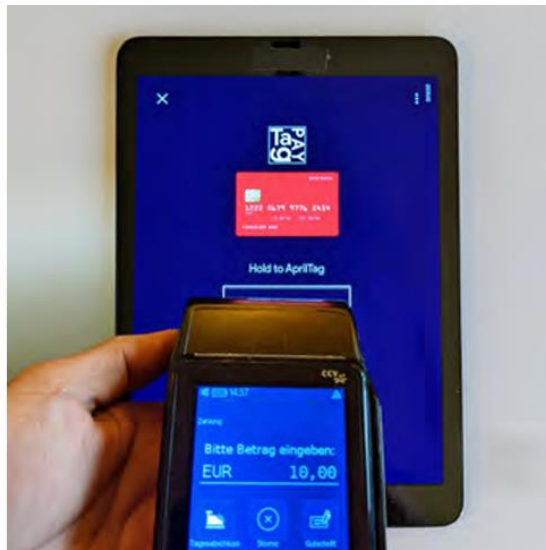
(b) Editing text with pen-like tangibles.



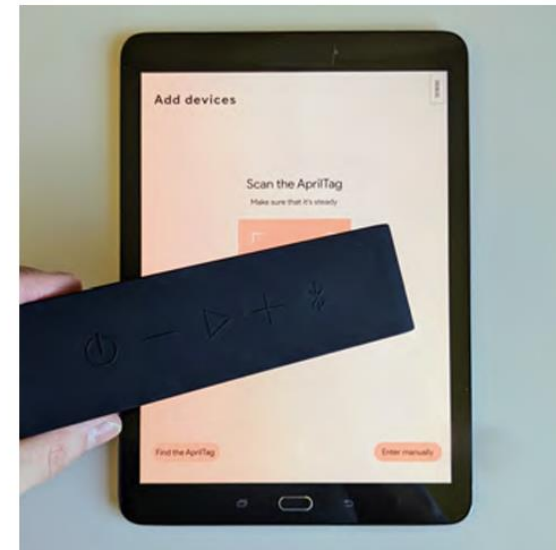
(c) Tower defense with tangible towers.



(d) Literacy learning with tangible letters.



(e) Fiducial marker to increase payment security.



(f) Scanning a fiducial marker instead of a QR-code to connect smart devices.

 [mimuc / super-resolution-for-fiducial-tangibles](#) Public

Notifications

Fork 1

Star 0

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

main 1 branch 0 tags

Go to file

Code

About

 **sven-mayer** Merge pull request #1 from MariusRusu97/main 9fdc609 on May 1 5 commits

📁 Data	Add code and data	2 months ago
📁 Eval	Add code and data	2 months ago
📁 Figs	Add code and data	2 months ago
📁 Models	Add code and data	2 months ago
📁 artk	Add code and data	2 months ago
📄 LICENSE	Initial commit	4 months ago
📄 README.md	Update README.md	2 months ago
📄 notebook.ipynb	Add code and data	2 months ago

☰ README.md

Deep Learning Super-Resolution Network Facilitating Fiducial Tangibles on Capacitive Touchscreens

Repository of the CHI'23 paper "Deep Learning Super-Resolution Network Facilitating Fiducial Tangibles on Capacitive Touchscreens"

Over the last few years, we have seen many approaches using tangibles to address the limited expressiveness of touchscreens. Mainstream tangible detection uses fiducial markers embedded in the tangibles. However, the coarse sensor size of capacitive touchscreens makes tangibles bulky, limiting their usefulness. We propose a novel deep-learning super-resolution network to facilitate fiducial tangibles on capacitive touchscreens better. In detail, our network super-resolves the markers enabling off-the-shelf detection algorithms to track tangibles reliably. Our network generalizes to unseen marker sets, such as AprilTag, ArUco, and ARToolKit. Therefore, we are not limited to a fixed number of distinguishable objects and do not require data collection and network training for new fiducial markers. With extensive evaluation, including real-world users and five showcases, we demonstrate the applicability of our open-source approach on commodity mobile devices and further highlight the potential of tangibles on capacitive touchscreens.

No description, website, or topics provided.

📖 Readme

📄 MIT license

☆ 0 stars

👁 9 watching

🍴 1 fork

Report repository

Releases

No releases published

Packages

No packages published

Contributors 2

 **sven-mayer** Sven Mayer

 **MariusRusu97**

Languages

Jupyter Notebook 98.1%

Python 1.4%

HTML 0.5%

<https://github.com/mimuc/super-resolution-for-fiducial-tangibles>

39

Sven Mayer

GAN Architectures

That's it? No.

The screenshot shows the GitHub repository for Keras-GAN, a public repository by eriklindernoren. The repository has 278 watchers, 3.2k forks, and 9k stars. It contains 185 commits and 1 branch. The repository is organized into a table of files and folders, each with a description and the time since the last update. The files include various GAN architectures like aae, acgan, bgan, bigan, ccgan, cgan, cogan, context_encoder, cyclegan, dcgan, discogan, dualgan, gan, infogan, lsgan, pix2pix, pixelda, sgan, srgan, wgan, wgan_gp, .gitignore, LICENSE, README.md, and requirements.txt. The repository also has a Readme, MIT license, and activity feed. The releases section shows no published releases, and the packages section shows no published packages. The contributors section shows 14 contributors, and the languages section shows Python at 99.2% and Shell at 0.8%.

File/Folder	Description	Updated
aae	Clean up in training loop	5 years ago
acgan	Update acgan.py	4 years ago
assets	WGAN (GP): Resolves #37. + clean up of handling input shapes of lat...	5 years ago
bgan	Update bgan.py	5 years ago
bigan	Clean up in training loop	5 years ago
ccgan	updated instance normalization import	4 years ago
cgan	Update cgan.py	5 years ago
cogan	Update cogan.py	5 years ago
context_encoder	Update context_encoder.py	5 years ago
cyclegan	updated instance normalization import	4 years ago
dcgan	Fix input shape of generator	5 years ago
discogan	updated instance normalization import	4 years ago
dualgan	Clean up in training loop	5 years ago
gan	removed hard-coded instances of self.latent_dim = 100	5 years ago
infogan	Clean up in training loop	5 years ago
lsgan	Update lsgan.py	5 years ago
pix2pix	updated instance normalization import	4 years ago
pixelda	updated instance normalization import	4 years ago
sgan	Fix image rescaling to [0,1]	5 years ago
srgan	updated instance normalization import	4 years ago
wgan	Fix image rescaling to [0,1]	5 years ago
wgan_gp	change input dim in critic to use latent_dim variable	4 years ago
.gitignore	Logo	5 years ago
LICENSE	Initial commit	6 years ago
README.md	Update README.md	3 years ago
requirements.txt	cleaned up requirements.txt	5 years ago

Conclusion

Generative Adversarial Network

- Generator
- Discriminator
- Generator tries to outperform the Discriminator to detect if it is a real or fake image

License

This file is licensed under the Creative Commons
Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Sven Mayer

